Guia rápido · Módulo 04: Git e GitHub

Resumo para consulta rápida. A explicação completa está nos slides e nos arquivos desta pasta.

Primeira configuração (uma vez por computador)

Ubuntu: `sudo apt update && sudo apt install git`. Confira com `git --version`.

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"   # o mesmo da conta do GitHub
git config --global init.defaultBranch main      # todo repositório nasce com main
git config --global pull.rebase false            # git pull junta com merge
git config --global core.editor "code --wait"    # opcional: mensagens no VS Code
git config --list                                # confere tudo
# rede com proxy (laboratório): uma linha só, vale para https:// também
git config --global http.proxy http://usuario:senha@IP:PORTA   # senha com @ ou #? use %40 e %23
```

No PC compartilhado, confira nome e e-mail antes de começar. O proxy usa as credenciais da rede da Lions, não as do GitHub; ao terminar a aula, remova-as com `git config --global --unset http.proxy`.

Repositório local (projeto novo)

```
git init                          # começa a versionar a pasta (uma vez, na pasta certa)
echo "# Lions Dev" > README.md
git status                        # o que mudou e em que branch você está
git diff                          # as linhas que mudaram (fora do palco)
git add .                         # põe tudo no palco
git diff --staged
git commit -m "adiciona README"
git log --oneline --graph         # histórico resumido (q para sair)
git show a1b2c3d                  # o que um commit mudou
```

Áreas: diretório → `git add` → palco → `git commit` → histórico. `git diff --staged` mostra o palco.

Repositório remoto (GitHub)

```
git remote add origin https://github.com/usuario/projeto.git
git remote -v                     # para onde o origin aponta
git remote set-url origin URL     # corrige a URL ("remote origin already exists")
git branch -M main
git push -u origin main           # primeira vez
git push                          # das próximas vezes
git fetch
git pull                          # = git fetch + git merge
```

`git fetch` baixa os commits sem mexer nos arquivos; `git pull` também os integra à branch atual.

> O GitHub **não aceita a senha** no terminal: use um **Personal Access Token** (Settings > Developer settings > Tokens (classic), com o escopo **repo**). Ao colar, nada aparece na tela; no Ubuntu, colar é `Ctrl+Shift+V`.

Clone e fork

```
git clone https://github.com/usuario/projeto.git
cd projeto
npm install
```

Clone copia para o computador. Use HTTPS na rede da Lions: SSH não funciona lá. A `node_modules` não vem junto; em projetos Node.js, recrie-a com `npm install`.

Fork copia para sua conta do GitHub, onde você pode dar push. Para contribuir no original: **Fork → clone do seu fork → alteração → Pull Request**. Confira seu `origin` com `git remote -v`; o botão **Sync fork** traz atualizações do original.

Branches e Pull Request

```
git branch                    # lista
git checkout -b raiz-quadrada # cria e muda (ou: git switch -c raiz-quadrada)
git checkout main             # volta (ou: git switch main)
git merge raiz-quadrada       # traz a branch para a atual
git branch -d raiz-quadrada   # apaga com verificação de integração
git branch -D raiz-quadrada   # força a exclusão
git push origin --delete raiz-quadrada   # apaga a branch no GitHub
```

O `-d` verifica se os commits estão na branch remota acompanhada (**upstream**, configurada pelo `push -u`); sem upstream, verifica a branch atual. Para apagar, saia da branch primeiro. Apagar a local não apaga a remota.

> No Pull Request: base `main` ← compare `sua-branch`. Para pedir revisão, o colega precisa ser colaborador (**Settings > Collaborators**) e aceitar o convite.

Conflito

```
<<<<<<< HEAD
console.log("versão da branch onde você está")
=======
console.log("versão que está chegando")
>>>>>>> raiz-quadrada
```

```
git status                    # lista os arquivos em "both modified"
git add arquivo.js            # depois de escolher o código e apagar as marcações
git commit -m "resolve conflito"
git merge --abort             # desiste e volta a como estava antes do merge
```

No VS Code: **Current** mantém HEAD, **Incoming** aceita a mudança que chega, **Both** mantém ambas. No `git pull`, o marcador final usa um hash.

Conflito no Pull Request? Na **sua branch**: `git pull origin main`, resolva, `git add .`, `git commit` e `git push`. O PR se atualiza sozinho.

Stash: guardar o trabalho pela metade

```
git stash          # guarda as mudanças não commitadas e limpa a pasta
git stash -u       # guarda também os arquivos novos (untracked)
git stash list     # o que está guardado (stash@{0} é o mais recente)
git stash pop      # recupera o último; tira da lista se não houver conflito
```

Se `stash pop` der conflito, resolva os arquivos; o stash permanece na lista.

Desfazendo

Situação	Comando
Descartar mudança fora do palco (não volta!)	`git restore arquivo.js`
Tirar do palco (mantém a mudança)	`git restore --staged arquivo.js`
Corrigir a mensagem do último commit	`git commit --amend -m "nova msg"`
Desfazer o último commit, mantendo as mudanças	`git reset --soft HEAD~1`
Desfazer um commit já enviado	`git revert a1b2c3d`

> `--amend` e `reset` só antes do push. Push rejeitado depois deles? **Não** use `git push --force`.

Versões antigas: `git checkout a1b2c3d` entra em **detached HEAD**, só para olhar/testar; `git checkout main` volta. Para recuperar só um arquivo: `git restore --source=a1b2c3d arquivo.js`; revise com `git diff`, faça `git add arquivo.js` e commit.

Fluxo do dia a dia

```
git checkout main && git pull       # 1. comece com a main atualizada
git checkout -b minha-tarefa        # 2. uma branch por tarefa
git add . && git commit -m "..."    # 3. commits pequenos
git push -u origin minha-tarefa     # 4. envia a branch
git checkout main && git pull       # 5. depois do merge, atualiza a main
git branch -d minha-tarefa          # 6. apaga a branch
```

Entre os passos 4 e 5: abra o **Pull Request**, peça revisão e faça o **merge no GitHub**.

.gitignore e README

No **.gitignore**, coloque `node_modules/` e `.env`, um por linha. Só ignora arquivos ainda não rastreados. O **README.md** explica o projeto: o que é, como instalar, como rodar e quem fez. Comandos vão dentro de um bloco com três crases, senão o GitHub junta as linhas.

Boas mensagens de commit

✔ `trata divisão por zero` · `adiciona README` ✘ `update` · `arrumei` · `asdf`

Armadilhas

- `git init` na pasta errada (na home inteira!) ou dentro de outro repositório. Confira com `pwd`.
- `src refspec main does not match any`: confira se já existe um primeiro commit e o nome da branch. Se ela for `master`, rode `git branch -M main`.
- `push` rejeitado (`fetch first`): alguém enviou antes. Faça `git pull`, resolva e envie de novo.
- O terminal abriu um editor? **nano**: `Ctrl+O`, `Enter`, `Ctrl+X`. **vim**: `i` para escrever; depois `Esc`, `:wq`, `Enter`.
- Mexeu na `main` sem criar a branch? `git switch -c nova-branch` leva as mudanças junto.
- Commitou `node_modules`? Tire do Git com `git rm -r --cached node_modules` e adicione ao `.gitignore`. Commitou `.env`? **Troque as senhas**: apagar o arquivo não apaga o histórico.